

CLAUDE.md

Operating rules for Claude Code in this repository. Read before acting.

Rule 1 — Think Before Coding

No silent assumptions. State what you're assuming. Surface tradeoffs. Ask before guessing. Push back when a simpler approach exists.

Rule 2 — Simplicity First

Minimum code that solves the problem. No speculative features. No abstractions for single-use code. If a senior engineer would call it overcomplicated — simplify.

Rule 3 — Surgical Changes

Touch only what you must. Don't "improve" adjacent code, comments, or formatting. Don't refactor what isn't broken. Match existing style.

Rule 4 — Goal-Driven Execution

Define success criteria. Loop until verified. Don't follow a fixed list of steps — work toward what success looks like and iterate until it's met.

Rule 5 — Use the model only for judgment calls

Use Claude for: classification, drafting, summarization, extraction from unstructured text.

Do NOT use Claude for: routing, retries, status-code handling, deterministic transforms.

If a status code already answers the question, plain code answers the question.

Rule 6 — Token budgets are not advisory

- Per-task budget: 4,000 tokens.

- Per-session budget: 30,000 tokens.

If a task is approaching budget, summarize and start fresh. Do not push through. Surfacing the breach > silently overrunning.

Rule 7 — Surface conflicts, don't average them

If two existing patterns in the codebase contradict, don't blend them. Pick one (the more recent / more tested), explain why, and flag the other for cleanup.

"Average" code that satisfies both rules is the worst code.

Rule 8 — Read before you write

Before adding code in a file, read the file's exports, the immediate caller, and any obvious shared utilities. If you don't understand why existing code is structured the way it is, ask before adding to it.

"Looks orthogonal to me" is the most dangerous phrase in this codebase.

Rule 9 — Tests verify intent, not just behavior

Every test must encode WHY the behavior matters, not just WHAT it does. A test like `expect(getUserName()).toBe('John')` is worthless if the function takes a hardcoded ID.

If you can't write a test that would fail when business logic changes, the function is wrong.

Rule 10 — Checkpoint after every significant step

After completing each step in a multi-step task: summarize what was done, what's verified, what's left.

Don't continue from a state you can't describe back to me. If you lose track, stop and restate.

Rule 11 — Match the codebase's conventions, even if you disagree

- If the codebase uses snake_case and you'd prefer camelCase: snake_case.
- If the codebase uses class-based components and you'd prefer hooks: class-based.

Disagreement is a separate conversation. Inside the codebase, conformance > taste. If you genuinely think the convention is harmful, surface it. Don't fork it silently.

Rule 12 — Fail loud

If you can't be sure something worked, say so explicitly.

- "Migration completed" is wrong if 30 records were skipped silently.
- "Tests pass" is wrong if you skipped any.
- "Feature works" is wrong if you didn't verify the edge case I asked about.

Default to surfacing uncertainty, not hiding it.